

## CHECK WHAT YOU LEARNT

**Check Yourself - Unit 4****A. Choose the correct answer**

1. A servo differs from an ordinary motor because it  
(a) spins faster (b) knows its own angle (c) needs no power (d) is smaller
2. The signal wire of an SG90 servo is usually  
(a) red (b) brown (c) orange (d) black
3. `lid.write(90)` tells the servo to  
(a) spin 90 times (b) turn to 90 degrees (c) wait 90 seconds (d) stop
4. The test `d > 0` is included because  
(a) 0 means very close (b) `pulseIn` returns 0 when no echo comes back (c) it looks neat (d) the servo needs it
5. The lid trembles while open. The most likely cause is  
(a) wrong code (b) the lid is too heavy (c) the sensor is dirty (d) a missing LED
6. A machine acting when nobody intended it to is called a  
(a) false trigger (b) short circuit (c) derived quantity (d) loop

**B. Fill in the blanks**

- 1 Never force a servo \_\_\_\_\_ round with your fingers.
- 2 The servo's ground must be \_\_\_\_\_ with the Arduino's ground.
- 3 In this project the lid opens when something is closer than \_\_\_\_\_ cm.
- 4 A part that moves or acts is called an \_\_\_\_\_.

**C. Answer in one or two lines**

- 1 Explain how a servo knows when it has reached the angle you asked for.
- 2 Name two things that could make your bin open when nobody is there.
- 3 Why is it worth watching a stranger use your machine?
- 4 Write down the `openDistance` you chose and why you chose it.

## UNIT 5

**Final Project: Obstacle-Avoiding Robot**

Sessions 12, 13, 14 and 15

Everything you have built so far has stayed on the desk. This one drives away. A battery, two motors, a driver board and the sensor you already know combine into a machine that crosses a room on its own.

By the end of this unit you will be able to:

- Explain how an H-bridge reverses a motor.
- Separate logic power from motor power while sharing a ground.
- Write reusable motion functions instead of repeating code.
- Debug a system where several subsystems interfere with each other.

## SESSION 12

one class

## Motors, the H-Bridge and Battery Safety

### Today you will

- Learn how four switches can reverse a motor.
- Read the L298N direction table.
- Agree the battery rules and the team roles.

To make a motor spin one way you send current through it in one direction. To reverse it you send the current the other way. An **H-bridge** is a clever arrangement of four electronic switches that lets a circuit do exactly that, on command, without anyone rewiring anything. It is called an H-bridge because in a circuit diagram the four switches and the motor form the shape of the letter H.

### The L298N: two switches decide which way a motor turns

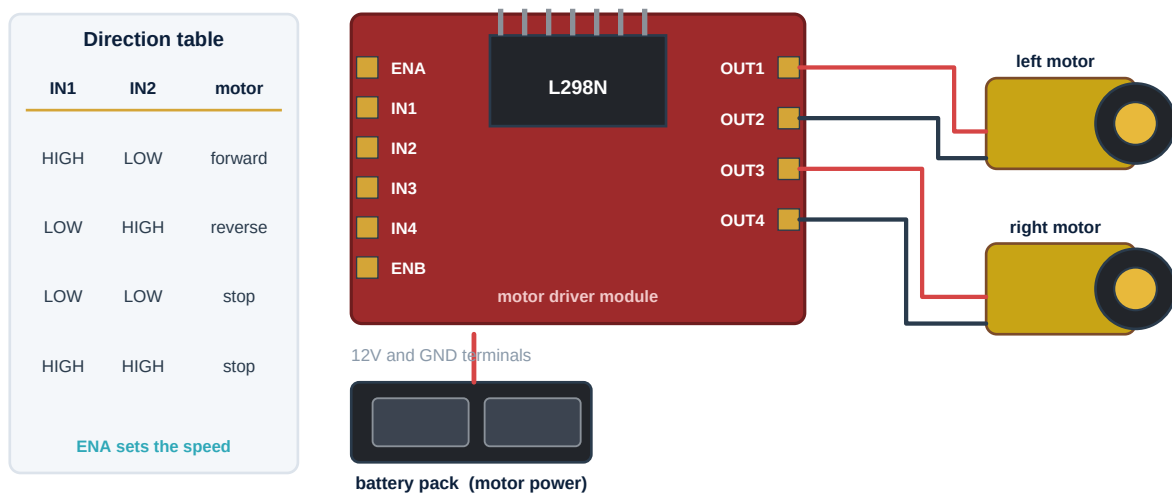


Figure 5.1 - IN pins choose the direction, EN pins choose the speed

The L298N contains two complete H-bridges, one per motor. Each motor needs two direction pins (IN1 and IN2 for the left, IN3 and IN4 for the right) and one enable pin (ENA, ENB) that controls speed. The direction table on the module tells you everything: one pin HIGH and the other LOW gives one direction, swap them for the other, and both the same means stop.

### Watch Out

The L298N has small jumper caps over the ENA and ENB pins. While those caps are fitted, the motors run at full speed and your speed control does nothing at all. **Remove both caps** before wiring ENA and ENB to the Arduino.

## 12.1 Two Power Supplies, One Ground

This is the single most important idea in the whole unit. Your robot has **two** sources of power. The USB or the battery feeds the Arduino, which needs very little. The battery pack feeds the L298N, which needs a great deal. These are separate on purpose - motors produce electrical noise that would otherwise reset the Arduino mid-drive.

But there is a catch. For the Arduino's signals to mean anything to the L298N, both must agree on what zero volts is. So you must run one wire from **Arduino GND to L298N GND**. Without it, everything looks correctly wired and absolutely nothing works. More school robots fail on this one missing wire than on any other fault.

| Role            | Responsible for                                    | Name  |
|-----------------|----------------------------------------------------|-------|
| Chassis builder | Mounting motors, wheels, caster and battery firmly | _____ |

| Role            | Responsible for                                        | Name  |
|-----------------|--------------------------------------------------------|-------|
| Wiring engineer | Every connection, checked twice, and the shared ground | _____ |
| Programmer      | Typing, uploading and tuning the turn timings          | _____ |
| Recorder        | The notebook: test runs, faults, and what fixed them   | _____ |

**SESSION 13**

one class

**Building the Drive Base****Today you will**

- Assemble the chassis, motors, wheels and battery.
- Wire the L298N and test all four directions.
- Write reusable motion functions.

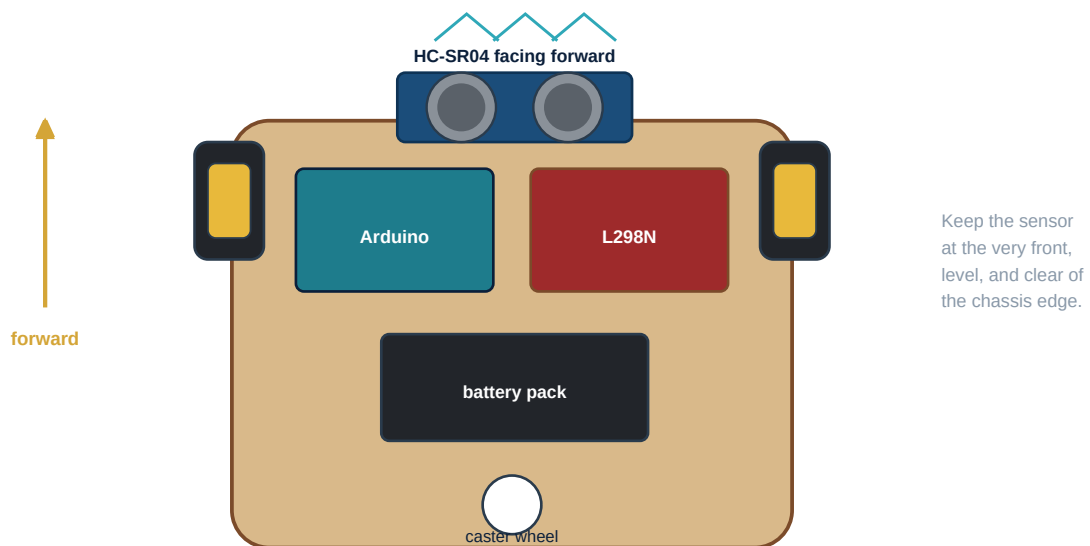
**Where everything sits on the robot**

Figure 5.2 - Plan the layout before you cut a single cable tie

- Mount both motors so the wheels are **parallel**. A few degrees out and the robot curves instead of going straight.
- Put the battery low and central - a high, off-centre battery makes the robot tip when it turns.
- Leave the caster wheel free to swivel. If it jams, the robot fights itself.
- Route wires so nothing can be caught by a turning wheel. Use tape or cable ties.

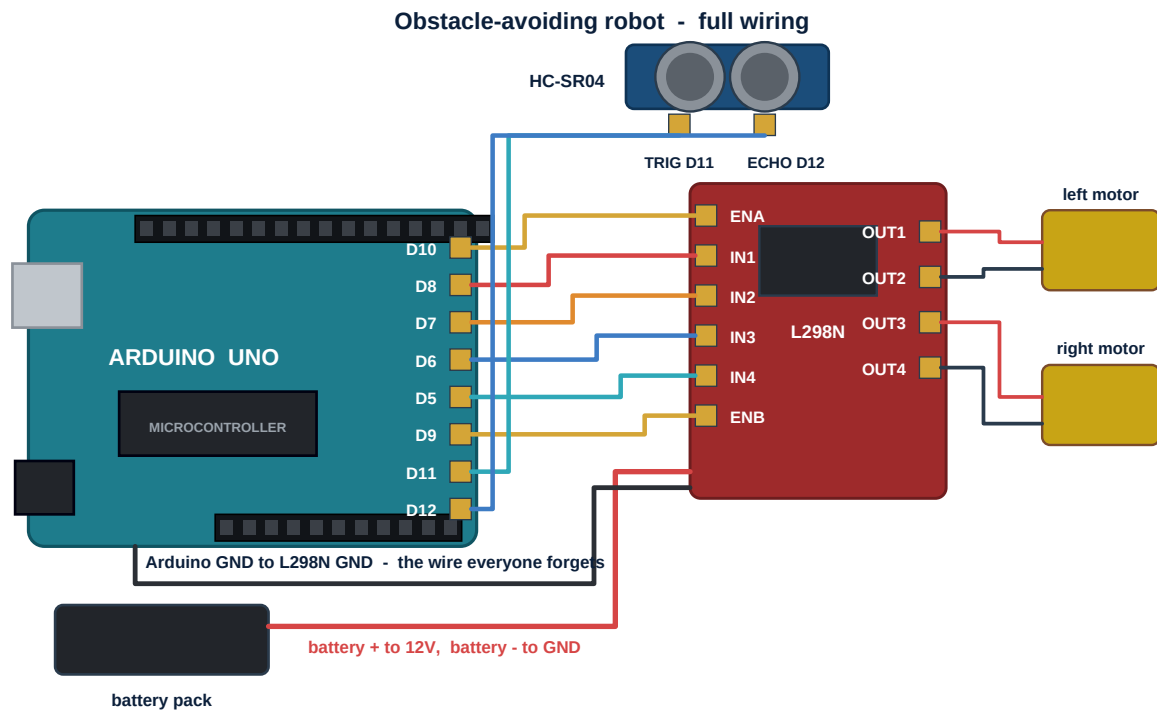


Figure 5.3 - Two power supplies, one shared ground

| Connection          | Goes to               | Why                                             |
|---------------------|-----------------------|-------------------------------------------------|
| L298N IN1 / IN2     | D8 / D7               | Left motor direction                            |
| L298N IN3 / IN4     | D6 / D5               | Right motor direction                           |
| L298N ENA / ENB     | D10 / D9              | Speed for each motor                            |
| L298N OUT1 / OUT2   | Left motor terminals  | Power to the left wheel                         |
| L298N OUT3 / OUT4   | Right motor terminals | Power to the right wheel                        |
| L298N 12V / GND     | Battery + / Battery - | Motor power                                     |
| <b>L298N GND</b>    | <b>Arduino GND</b>    | <b>The shared ground. Never leave this out.</b> |
| HC-SR04 TRIG / ECHO | D11 / D12             | Obstacle detection                              |

**DriveBaseTest.ino - test the base before adding the sensor**

```

int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 150;           // 0 to 255. Start slow.

void setup() {
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
}

void forward() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void back() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, LOW); digitalWrite(in2, HIGH);
  digitalWrite(in3, LOW); digitalWrite(in4, HIGH);
}

```

```
void turnRight() {           // left wheel forward, right wheel back
  analogWrite(ena, speed);   analogWrite(enb, speed);
  digitalWrite(in1, HIGH);   digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);    digitalWrite(in4, HIGH);
}

void stopMotors() {
  analogWrite(ena, 0);   analogWrite(enb, 0);
}

void loop() {             // a simple test of all four
  forward();   delay(1500);
  back();     delay(1500);
  turnRight(); delay(800);
  stopMotors(); delay(2000);
}
```

#### Watch Out

If a wheel spins the wrong way, do not rewrite the code. Simply swap that motor's two wires at the L298N terminals. It is faster and it is what a professional would do.

### SESSION 14

one class

## Adding the Eyes and the Algorithm

#### Today you will

- Mount the ultrasonic sensor at the front.
- Write the avoidance algorithm.
- Run the robot for the first time.

The robot's decision is simpler than people expect. Drive forward. Keep looking ahead. If something is closer than a set distance, stop, reverse a little to make room, turn, and carry on. Written as a flowchart before it is written as code, it takes five boxes.

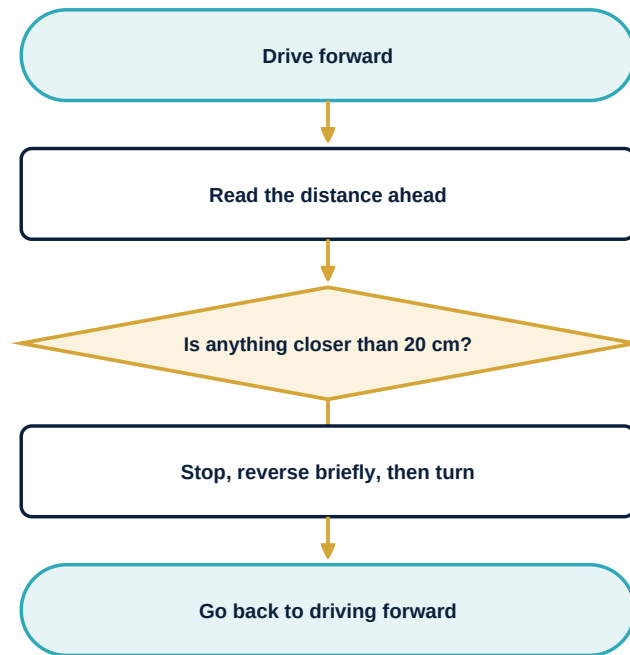


Figure 5.4 - The avoidance algorithm, in five steps

**ObstacleAvoidingRobot.ino**

```

int trig = 11, echo = 12;
int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 150;
int safeDistance = 20;          // centimetres

void setup() {
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
  pinMode(trig, OUTPUT); pinMode(echo, INPUT);
  Serial.begin(9600);
  delay(2000);                  // time to put the robot down and step back
}

int readDistance() {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long t = pulseIn(echo, HIGH, 25000); // give up after 25 ms
  int cm = t * 0.034 / 2;
  if (cm == 0) { cm = 400; }           // no echo means nothing is near
  return cm;
}

void forward() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void back() {
  analogWrite(ena, speed); analogWrite(enb, speed);

```

```

    digitalWrite(in1, LOW);    digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);    digitalWrite(in4, HIGH);
}

void turnRight() {
    analogWrite(ena, speed);    analogWrite(enb, speed);
    digitalWrite(in1, HIGH);    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);     digitalWrite(in4, HIGH);
}

void stopMotors() {
    analogWrite(ena, 0);    analogWrite(enb, 0);
}

void loop() {
    int d = readDistance();
    Serial.println(d);

    if (d < safeDistance) {
        stopMotors();    delay(200);
        back();          delay(400);    // make room to turn
        stopMotors();    delay(200);
        turnRight();     delay(500);    // tune this number on your own floor
        stopMotors();    delay(200);
    } else {
        forward();
    }

    delay(60);
}

```

Two details deserve attention. The third argument to `pulseIn(echo, HIGH, 25000)` is a **timeout**: if no echo arrives within 25 milliseconds, give up rather than freezing the robot mid-drive. And the line turning a 0 into 400 says, in effect, *hearing nothing means nothing is there* - which is true, and which stops the robot spinning on the spot in an open room.

## SESSION 15

one class

# Arena Testing and Your Demonstration

### Today you will

- Tune the turn timing on the actual floor.
- Pass every acceptance test.
- Present your robot in three minutes.

Your robot will behave differently on smooth tiles and on rough concrete, and differently again as the battery drains. Tuning is not cheating - it is the job. Work through the fault table before you start changing numbers at random.

| What you see                             | Most likely cause                            | What to do                                          |
|------------------------------------------|----------------------------------------------|-----------------------------------------------------|
| Nothing happens at all                   | Missing Arduino-to-L298N ground wire         | Add it. This is the number one fault.               |
| Motors run flat out, speed ignored       | The ENA and ENB jumper caps are still fitted | Remove both caps                                    |
| Robot curves instead of driving straight | One motor is faster than the other           | Use a slightly lower speed value on the faster side |

| What you see                         | Most likely cause                   | What to do                                           |
|--------------------------------------|-------------------------------------|------------------------------------------------------|
| Robot spins forever in an empty room | A 0 reading treated as very close   | Keep the line that converts 0 into 400               |
| Robot resets whenever motors start   | Motor noise, or a weak battery      | Charge or replace the cells; check the shared ground |
| Turns too little or too much         | Turn delay not tuned for this floor | Adjust the delay after turnRight in steps of 100     |

| Acceptance test                                     | Result | Tick |
|-----------------------------------------------------|--------|------|
| Drives straight for 2 metres with reasonable drift  | _____  | ■    |
| Stops before touching a wall, every time            | _____  | ■    |
| Turns and continues without human help              | _____  | ■    |
| Crosses a 3 x 3 m arena with 5 obstacles, no rescue | _____  | ■    |
| Runs for 3 minutes without resetting                | _____  | ■    |

## 15.1 Your Three-Minute Demonstration

- The problem** - one sentence on what your machine does and why that is useful.
- How it senses** - hold up the HC-SR04 and explain the echo in your own words.
- How it decides** - show the line containing safeDistance and say what the number means.
- How it acts** - explain the H-bridge in one sentence, then run the robot live.
- What went wrong** - name one fault you found and fixed. Judges value this most.
- What next** - one honest improvement you would make with more time.

### Challenge

Mount the HC-SR04 on the SG90 servo. When the robot meets an obstacle, stop, sweep the sensor left and right, compare the two distances, and turn towards whichever side has more space. This is how real robots choose a path rather than always turning the same way.

| New word      | What it means                                                          |
|---------------|------------------------------------------------------------------------|
| H-bridge      | Four switches that let a circuit reverse a motor on command.           |
| Motor driver  | A board that switches heavy motor current on a small signal's command. |
| Shared ground | The wire that lets two power supplies agree on what zero volts means.  |
| Timeout       | Giving up on waiting so a program is never stuck.                      |
| Algorithm     | A clear, ordered set of steps a machine follows to solve a problem.    |